

Wind Turbine Fault Detection Using SCADA Data: A Machine Learning Approach to Predictive Maintenance

Abstract

Wind turbines are critical assets in modern renewable energy systems, yet unexpected failures can result in costly downtime, expensive repairs, and lost power generation. This project develops an end-to-end machine learning pipeline to predict near-term turbine failures using multivariate SCADA time-series data. The problem is formulated as a binary classification task, where each timestamp is labeled based on whether a failure event occurs within a future time window (24, 48, or 72 hours).

The methodology integrates data preprocessing, temporal feature engineering, event-based labeling, and supervised learning models within a modular and reproducible pipeline. Logistic Regression, Random Forest, and XGBoost models are evaluated under severe class imbalance using precision-recall metrics and threshold optimization.

Results show that turbine failures are best characterized as gradual degradation processes rather than discrete events, with predictive signals distributed over time. While absolute performance metrics remain modest due to the inherent difficulty of the problem, the models consistently identify meaningful early warning patterns aligned with known physical failure mechanisms. These findings demonstrate the feasibility of data-driven predictive maintenance systems for renewable energy infrastructure and highlight key considerations for operational deployment.

1. Introduction and Problem Statement

Wind turbines play a central role in the transition toward renewable energy, but their reliability remains a critical operational concern. Unexpected component failures can lead to prolonged downtime, costly repairs, and reduced energy production. Traditional maintenance strategies are often reactive or based on fixed schedules, which do not fully leverage the extensive telemetry data collected by modern Supervisory Control and Data Acquisition (SCADA) systems.

This project addresses the challenge of developing a data-driven predictive maintenance system capable of identifying early warning signals of turbine failure. Specifically, the objective is to predict whether a turbine will experience a failure within a defined future time window using historical SCADA data.

Rather than attempting to predict the exact moment of failure, the problem is formulated as a binary classification task, where each timestamp is labeled based on whether a failure event occurs within a future horizon (e.g., 24–72 hours). This formulation emphasizes early detection

of degradation, which is more valuable in real-world maintenance scenarios than precise failure timing.

A key insight guiding this work is that turbine failures are not isolated events but the result of **gradual degradation in system behavior**. As such, effective prediction requires capturing temporal dynamics rather than relying solely on instantaneous sensor values. This perspective shapes the entire modeling approach, from feature engineering to evaluation design.

2. Dataset and Methodology

2.1 Dataset Description

The dataset used in this project consists of publicly available wind turbine SCADA data collected from multiple wind farms and published through Zenodo as Wind Turbine SCADA Data for Early Fault Detection (Kasimov, 2024). It includes multivariate, time-stamped measurements representing turbine operating conditions over time. Each observation corresponds to a snapshot of system behavior and is associated with event metadata indicating failure or anomaly occurrences.

The dataset includes several categories of features:

- Environmental variables (e.g., wind speed, ambient temperature)
- Mechanical variables (e.g., rotor speed, generator speed)
- Electrical variables (e.g., power output, voltage)
- Thermal variables (e.g., gearbox and bearing temperatures)

The data is inherently time-series in nature, with observations collected at regular intervals for multiple turbines across different wind farms.

Several challenges are present in the dataset:

- **Severe class imbalance**, with failure events representing a small fraction of observations
- **Cross-farm inconsistencies**, including differences in sensor naming, availability, and calibration
- **Missing data and irregular time gaps**
- **Diffuse failure signals**, where degradation occurs gradually rather than abruptly

These characteristics significantly influence the design of the preprocessing pipeline, feature engineering strategy, and evaluation methodology.

These limitations also shaped the scope of the analysis. Because sensor names and availability differ across farms, features could not be combined solely by matching column names; they had to be harmonized according to physical meaning. Some useful variables were unavailable or had to be derived, and missing values and irregular time gaps required conservative preprocessing. The small number of failure-labeled observations further limited achievable model performance and made precision-recall evaluation more appropriate than accuracy-based evaluation.

2.2 Methodology

The methodology follows a structured data mining pipeline that integrates data processing, feature engineering, labeling, modeling, and evaluation into a unified and reproducible workflow.

To support reproducibility and maintainability, the full pipeline was implemented using a modular codebase organized as a Python package. This structure separates data processing, feature engineering, and modeling components into distinct modules, enabling consistent experimentation and scalable execution. While the final notebook consolidates the full workflow for submission, this modular design reflects best practices in applied machine learning system development.

2.2.1 Data Processing and Preprocessing

Data preprocessing is designed to ensure data quality while preserving temporal integrity. Raw SCADA data is processed at the turbine level to avoid memory constraints and maintain chronological ordering.

Key preprocessing steps include:

- Handling missing values through selective feature retention and cleaning
- Removing or excluding features with excessive missingness
- Harmonizing sensor definitions across wind farms to ensure consistency
- Preserving strict temporal ordering of observations

This turbine-level processing approach allows the pipeline to scale to large datasets while maintaining alignment between sensor data and failure events.

Cross-Farm Feature Availability and Harmonization Overview			
Standardized modeling feature	Wind Speed	A	A
	Active Power	A	A
	Gen Speed	A	A
	Nacelle Temp	A	A
	Gearbox Oil Temp	A	A
	Generator Temp	A	A
	Hub Temp	A	A
	Hydraulic Temp	A	U
	Vibration Raw	A	A
	Yaw Error	A	A
	Temp Delta Gearbox	D	D
	Power Efficiency	D	D
	Vibration Magnitude	D	D
		Farm A	Farm B
Wind farm			
Available Derived Unavailable			

Figure 1. Cross-farm feature availability and harmonization overview. Blue cells indicate directly available features via mapping, orange cells indicate derived features, and gray cells indicate unavailable features.

This overview supports the preprocessing decision to harmonize features by physical meaning rather than relying only on raw column names.

2.2.2 Feature Engineering

Feature engineering focuses on capturing temporal degradation patterns, which are essential for failure prediction.

Rather than relying on raw sensor values alone, the following transformations are applied:

- Rolling statistics (mean, standard deviation)
- Lag features to capture recent history
- Rate-of-change metrics
- Volatility-based features to measure instability

These features allow the model to detect trends, fluctuations, and evolving system behavior over time.

Additionally, feature design is guided by physical interpretability, prioritizing signals known to be associated with turbine degradation, such as vibration, temperature changes, and operational instability.

2.2.3 Labeling Strategy

The prediction task is defined using an event-based labeling approach. Each timestamp is labeled based on whether a failure event occurs within a future time window.

Three prediction horizons are evaluated:

- 24 hours
- 48 hours
- 72 hours

To construct these labels, each timestamp is assigned a positive label if a failure event occurs within the corresponding future window. This results in multiple overlapping pre-failure regions for different prediction horizons.

To prevent ambiguity and data leakage, buffer zones are introduced around failure events. Importantly, the pre-event buffer is applied immediately before the earliest prediction window (e.g., the start of the 72-hour window), rather than directly before the failure event itself. This ensures that:

- Positive labels represent meaningful pre-failure conditions without contamination from transition periods
- Negative samples are not influenced by early degradation signals that overlap with labeled regions

In addition, a post-event buffer is applied after the failure event to remove observations that may reflect abnormal system behavior following the event.

This labeling strategy is illustrated conceptually in Figure 2, which shows how prediction windows and buffer zones are defined relative to failure events.

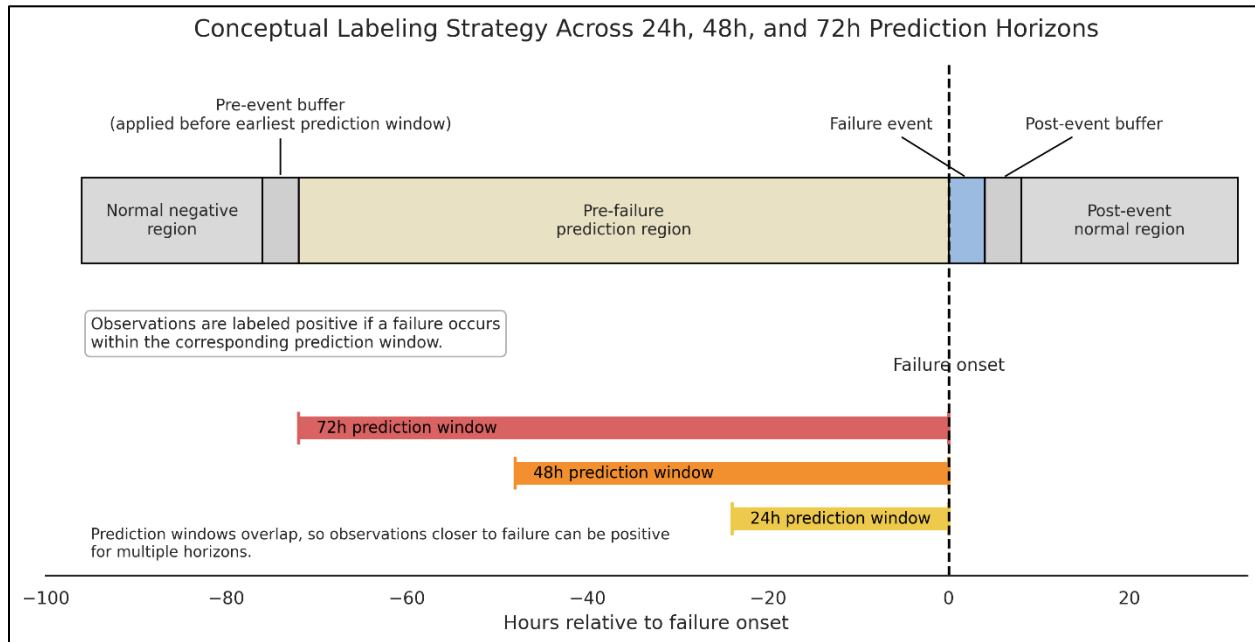


Figure 2. Conceptual event-based labeling strategy for 24-hour, 48-hour, and 72-hour prediction horizons.

Overall, this approach shifts the problem from predicting exact failure timing to identifying broader degradation patterns, which is more aligned with real-world maintenance objectives.

2.2.4 Modeling Approach

Three supervised learning models are evaluated:

- **Logistic Regression**, serving as an interpretable baseline
- **Random Forest**, capturing nonlinear relationships and feature interactions
- **XGBoost**, a gradient boosting model known for strong performance on structured data

These models were selected to provide a balance between interpretability and predictive power. Logistic Regression offers insight into linear relationships, while tree-based models are better suited for capturing complex interactions in high-dimensional feature spaces.

2.2.5 Evaluation Strategy

Due to the severe class imbalance, traditional accuracy is not an appropriate metric. Instead, evaluation focuses on:

- Precision (reliability of positive predictions)
- Recall (ability to detect failure events)
- F1 score (balance between precision and recall)

To further improve model usability, threshold optimization is applied. Rather than using a default classification threshold, thresholds are selected based on validation performance to achieve a more effective precision–recall balance.

This approach ensures that model evaluation aligns with real-world operational constraints, where both missed failures and excessive false alarms carry significant costs.

While probability calibration techniques (e.g., Platt scaling or isotonic regression) could improve the alignment between predicted probabilities and true event likelihood, they were not applied in this project. Given the scope of the course and the focus on classification performance, threshold optimization provides a practical and commonly used approach to achieving an effective precision–recall balance in imbalanced classification problems.

3. Results and Analysis

Before evaluating model performance, it is important to understand the degree of class imbalance present in the dataset. Failure events represent only a small fraction of total observations, creating a highly imbalanced classification problem.

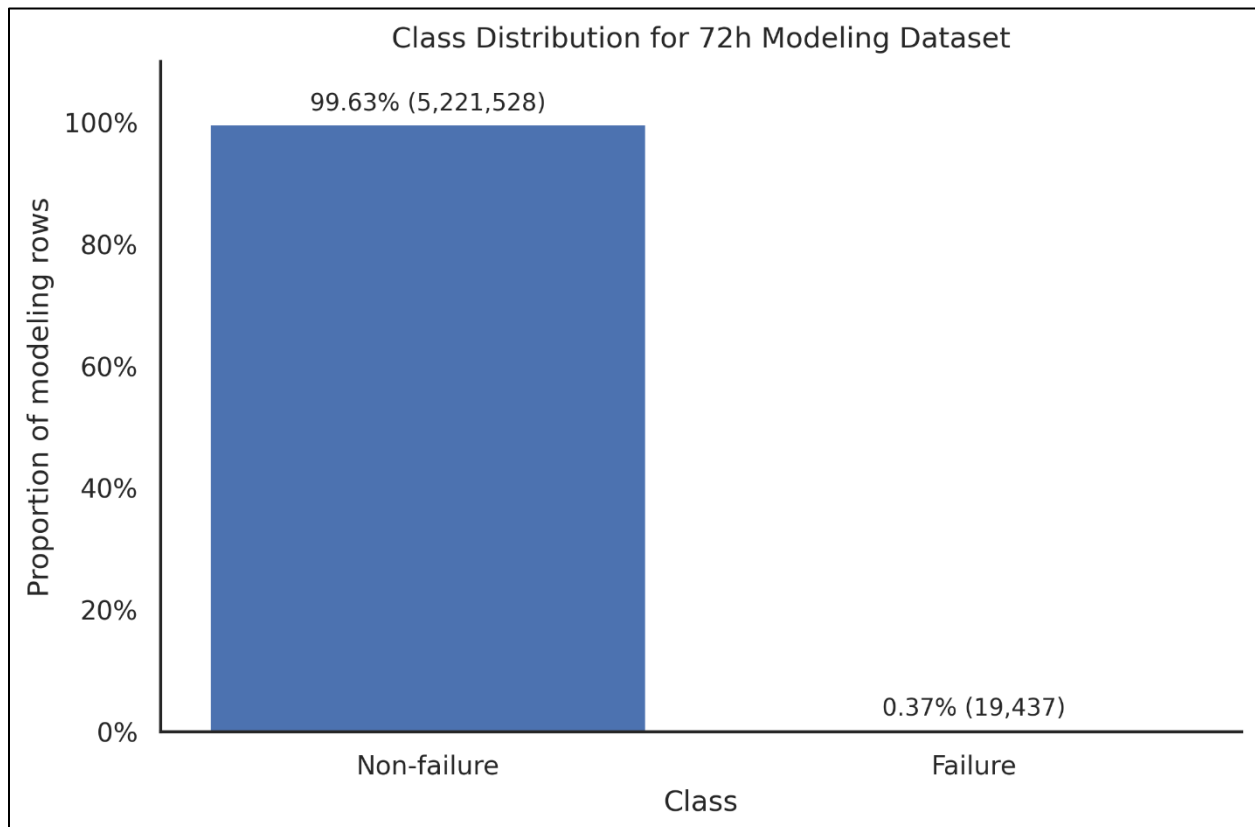


Figure 3. Class distribution for the 72-hour modeling dataset. Failure-labeled rows represent only 0.37% of observations, motivating precision-recall-based evaluation.

This imbalance significantly influences model behavior and evaluation metrics. In particular, it limits achievable precision and recall, and necessitates the use of precision–recall-based evaluation rather than accuracy.

3.1 Model Performance

Among the evaluated models, XGBoost consistently achieved the strongest performance across all prediction windows. While absolute performance metrics are modest, this is expected given the rarity of failure events, which represent only a small fraction of total observations and create a highly imbalanced classification problem.

To provide a comprehensive comparison, Table 1 summarizes performance across all three models for the 72-hour prediction window.

Table 1. Model performance comparison for the 72-hour prediction horizon.

Model	Precision	Recall	F1
XGBoost	0.127	0.135	0.131
Logistic Regression	0.031	0.703	0.059
Random Forest	0.065	0.032	0.043

XGBoost outperforms both Logistic Regression and Random Forest in terms of overall F1 score, indicating its ability to better capture nonlinear relationships and temporal feature interactions.

3.2 Prediction Window Comparison

Model performance is broadly consistent across 24-hour, 48-hour, and 72-hour prediction windows. This indicates that predictive signals are not confined to a narrow time boundary but are distributed over a longer period leading up to failure.

To illustrate this, Table 2 summarizes XGBoost performance across the three prediction horizons.

Table 2. XGBoost performance comparison across prediction horizons.

Window	Precision	Recall	F1
24h	0.125	0.132	0.128
48h	0.127	0.106	0.116
72h	0.127	0.135	0.131

The relatively small differences in precision, recall, and F1 score across windows suggest that the underlying predictive signal remains stable regardless of the specific horizon definition. In other words, the model is learning patterns associated with **ongoing system degradation**, rather than signals tied to a precise failure time.

This finding has important implications for both modeling and interpretation. First, it reinforces the idea that failure prediction should be framed as an early warning problem rather than a point

prediction task. Second, it suggests that the choice of prediction window primarily affects how predictions are interpreted operationally, rather than what the model is able to learn from the data.

From an operational perspective, shorter windows (e.g., 24 hours) may support more immediate intervention, while longer windows (e.g., 72 hours) provide earlier warning at the cost of increased uncertainty. The similarity in model performance across these windows indicates that this tradeoff is driven more by decision-making needs than by differences in predictive signal strength.

3.3 Threshold Behavior

Threshold optimization plays a critical role in achieving usable model performance. Default classification thresholds result in excessive false positives, while optimized thresholds provide a more effective balance between precision and recall.

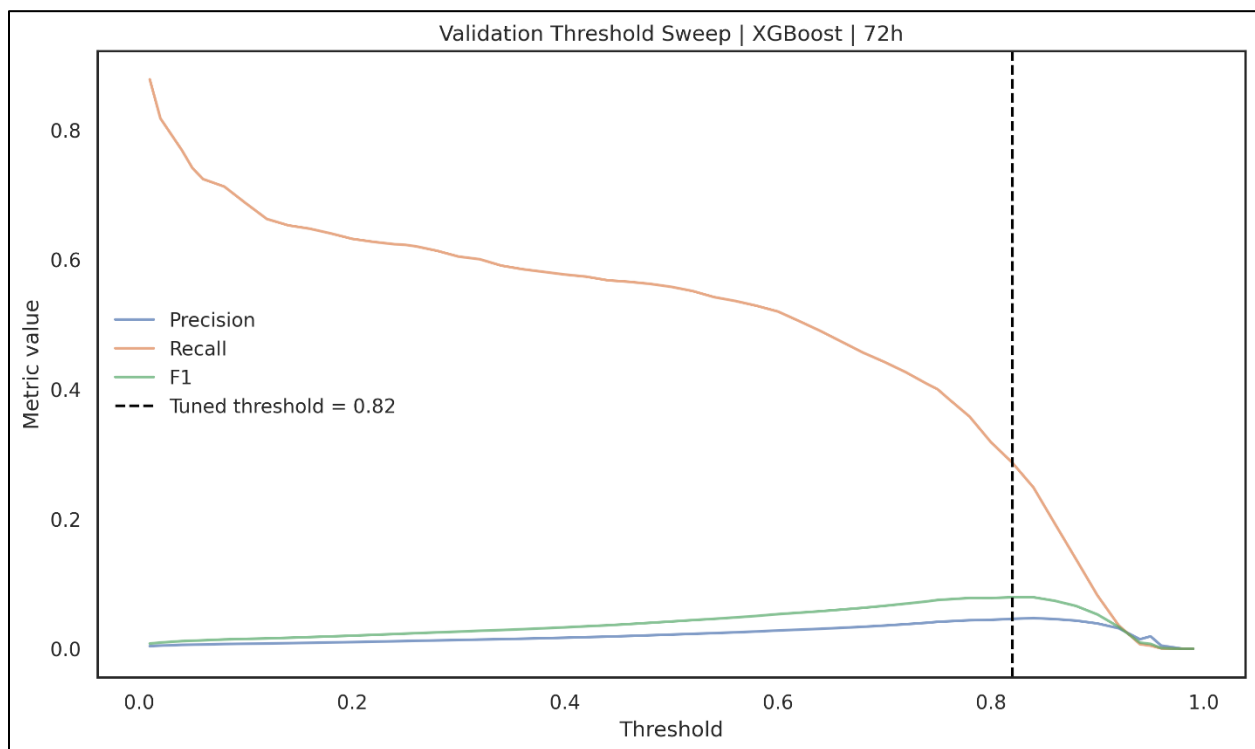


Figure 4. Validation threshold sweep for the XGBoost 72-hour model. The selected threshold balances precision and recall under severe class imbalance.

This behavior suggests that the raw model scores should be interpreted as ranking signals rather than calibrated probabilities. As a result, threshold tuning is necessary to convert continuous scores into operational alerts rather than relying on default classification thresholds.

3.4 Temporal Prediction Behavior

Model predictions exhibit strong temporal structure, with elevated probabilities occurring prior to failure events and persisting over multiple time steps.

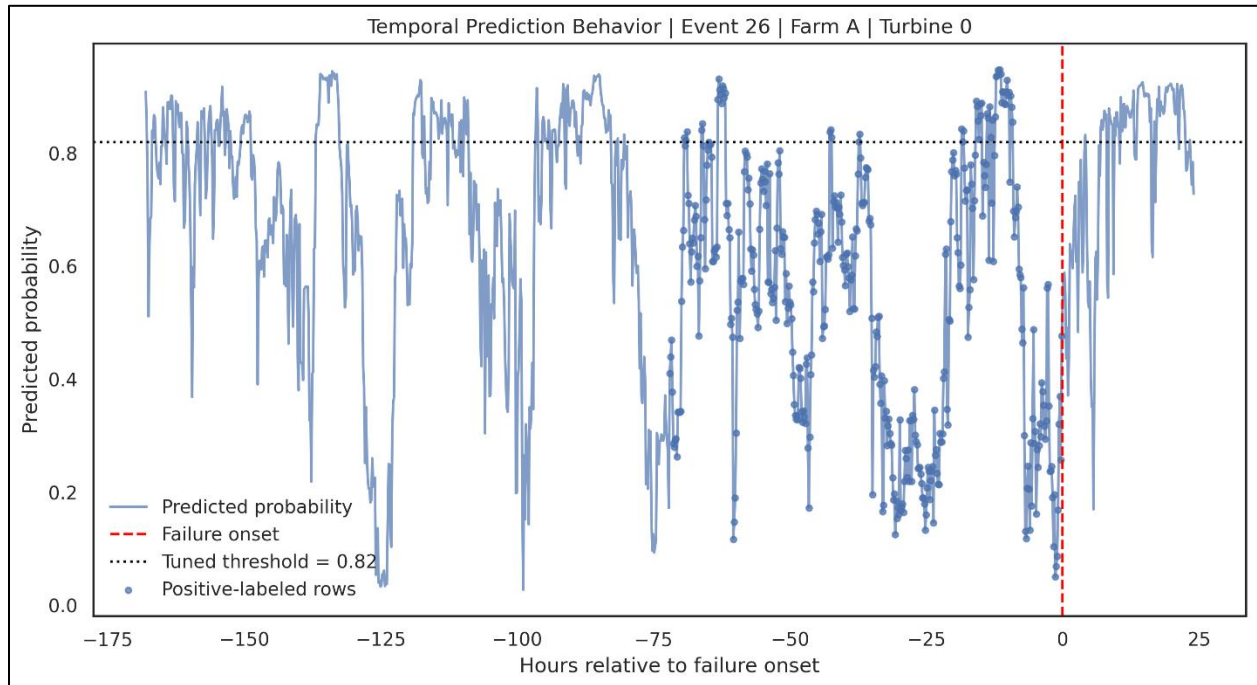


Figure 5. Example temporal prediction behavior around a failure event. The plot illustrates how model scores vary over time relative to failure onset.

This clustering behavior suggests that failures are preceded by sustained degradation rather than isolated anomalies, and that models are capturing temporally correlated signals indicative of evolving system instability.

3.5 Feature Importance

Feature importance analysis reveals that the most influential predictors are physically meaningful variables, including vibration signals, temperature dynamics, and operational efficiency measures.

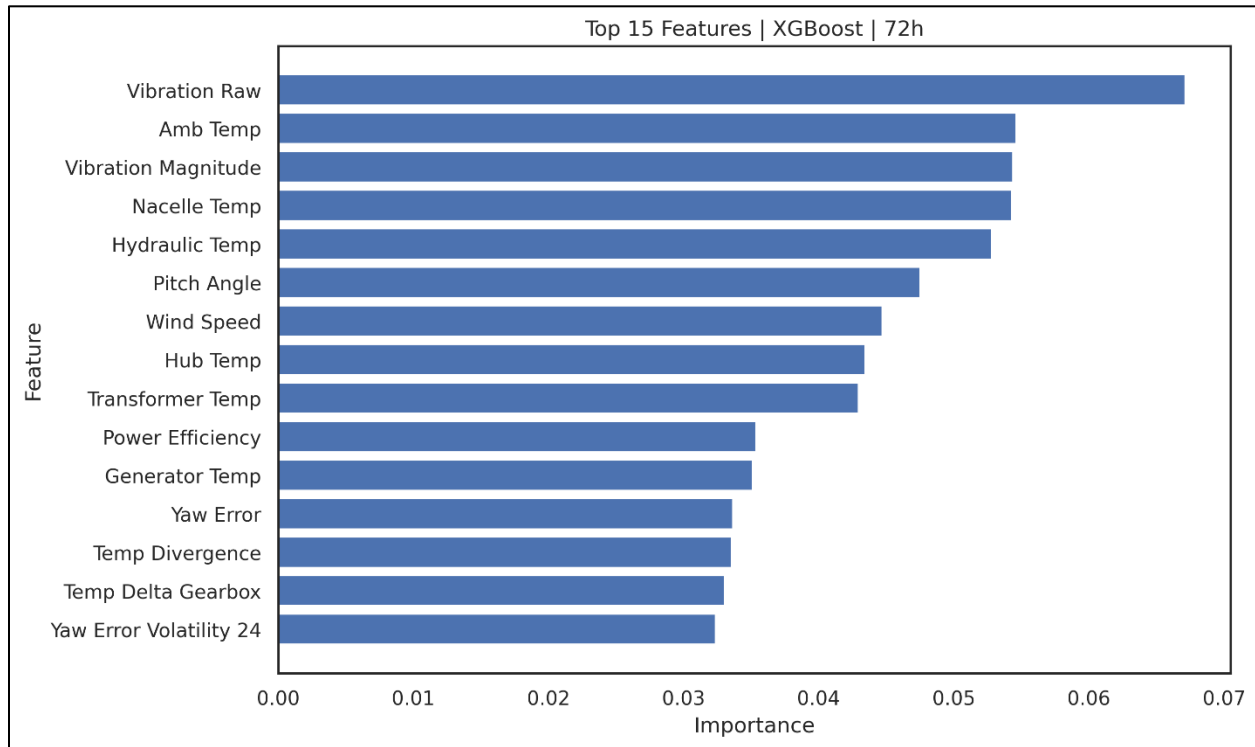


Figure 6. Top 15 XGBoost feature importances for the 72-hour prediction horizon.

The top-ranked predictors support the interpretation that the model is learning physically meaningful degradation signals. Vibration-related variables can reflect mechanical imbalance or component wear, temperature variables can indicate abnormal thermal stress in drivetrain or generator systems, and power-efficiency measures capture whether the turbine is producing expected output under observed operating conditions. Yaw-error and volatility-based features further suggest that instability in turbine orientation and short-term operating behavior may provide early evidence of developing faults.

Feature importance is primarily derived from the XGBoost model, which provides the most stable and interpretable importance measures for this problem. While other models also provide importance metrics, they are less reliable in capturing complex nonlinear relationships present in the data.

The consistency of these features across experiments indicates that the model is capturing real system behavior rather than spurious correlations.

4. Discussion

The results of this analysis highlight several important insights about wind turbine failure prediction and time-series modeling more broadly.

First, failure prediction is fundamentally a temporal degradation problem. Models must capture how system behavior evolves over time rather than relying on static measurements. This explains the strong performance of temporal features such as rolling statistics and lagged variables.

Second, predictive signals are diffuse rather than localized, appearing well before the defined prediction window and persisting over time. This makes precise failure timing difficult but supports the feasibility of early warning systems.

Third, the observed precision–recall tradeoff reflects real-world operational constraints. While high recall is achievable, it often comes at the cost of excessive false positives. Effective systems must balance these competing objectives to remain practical for maintenance teams.

Finally, the relatively low performance metrics should be interpreted in the context of the problem. Wind turbine failure prediction involves rare events, noisy data, and complex system dynamics. In such settings, even modest predictive performance represents meaningful signal detection rather than random noise.

The low precision and recall values are also a direct consequence of the extreme class imbalance in the modeling dataset. When failure-labeled rows account for only 0.37% of observations, even a model that detects meaningful risk patterns will produce many false positives relative to true positives. In this setting, low absolute scores do not necessarily imply that the model is uninformative; rather, they reflect the difficulty of identifying rare degradation events from noisy operational data.

XGBoost likely outperformed Logistic Regression and Random Forest because it is well suited to nonlinear, structured data problems with interacting features. Logistic Regression provides a useful baseline, but it is limited in its ability to represent nonlinear degradation patterns. Random Forest can capture nonlinear relationships, but the boosted-tree approach used by XGBoost more effectively combines weak signals across many engineered temporal features, which is important when failure precursors are subtle and distributed over time.

Threshold tuning is operationally important because the model output is most useful as a prioritization signal rather than a fully calibrated probability of failure. Maintenance teams would need to choose thresholds based on the relative cost of missed failures versus unnecessary inspections. A lower threshold may be appropriate when missing a failure is very costly, while a higher threshold may be preferred when false alarms create excessive maintenance burden. This reinforces that model deployment would require both technical validation and operational decision rules.

Alternative approaches, such as sequence-based time-series models (e.g., LSTM or transformer architectures), could potentially capture temporal dependencies more explicitly. However, these methods require larger volumes of clean, well-labeled sequential data and introduce additional modeling complexity. Given the dataset characteristics and project scope, the feature-based classification approach used in this project provides a practical and interpretable solution aligned with common industry practice.

5. Conclusion and Recommendations

This project demonstrates that machine learning can be effectively applied to wind turbine SCADA data to detect early warning signals of failure. The results show that failure prediction is feasible when framed as a temporal degradation problem and supported by appropriate feature engineering and evaluation strategies.

However, several challenges remain. Predictive signals are distributed over time, making it difficult to align predictions with precise operational decision points. Additionally, class imbalance and data variability limit achievable performance.

Future work should focus on improving the operational usability of model outputs, including prediction smoothing, event-based evaluation metrics, and probability calibration. Enhancing model generalization across turbines and wind farms also represents an important area for further development.

Overall, this work highlights both the potential and the limitations of data-driven predictive maintenance systems in real-world engineering applications.

References

Kasimov, A. (2024). Wind turbine SCADA data for early fault detection [Data set]. Zenodo. <https://doi.org/10.5281/zenodo.10958775>

AI Use Disclosure

Generative AI tools were used as a development aid to improve efficiency, code quality, and clarity of written communication. AI assistance included support for drafting and refining documentation, improving code organization, and providing guidance on software architecture decisions.

All problem formulation, data processing, feature engineering, model development, experimentation, and interpretation of results were performed independently. AI tools were used to enhance clarity and reduce errors but did not replace independent technical work or decision-making.